



ELSEVIER

Information Processing Letters 80 (2001) 279–285

Information
Processing
Letters

www.elsevier.com/locate/ipl

A fast and practical bit-vector algorithm for the Longest Common Subsequence problem

Maxime Crochemore^{a,1}, Costas S. Iliopoulos^{b,c,2}, Yoan J. Pinzon^{b,*,3}, James F. Reid^{b,d,4}

^a Institut Gaspard-Monge, Université de Marne-la-Vallée, F-77454 Marne-la-Vallée Cedex 2, France

^b Department Computer Science, King's College London, London WC2R 2LS, UK

^c School of Computing, Curtin University of Technology, GPO Box 1987 U, Perth 6845, Western Australia

^d LNCIB, Area Science Park, Padriciano 99, Trieste, 34102, Italy

Received 2 February 2001; received in revised form 9 March 2001

Communicated by L. Boasson

Abstract

This paper presents a new practical bit-vector algorithm for solving the well-known Longest Common Subsequence (LCS) problem. Given two strings of length m and n , $n \geq m$, we present an algorithm which determines the length p of an LCS in $O(nm/w)$ time and $O(m/w)$ space, where w is the number of bits in a machine word. This algorithm can be thought of as column-wise “parallelization” of the classical dynamic programming approach. Our algorithm is very efficient in practice, where computing the length of an LCS of two strings can be done in linear time and constant (additional/working) space by assuming that $m \leq w$. © 2001 Elsevier Science B.V. All rights reserved.

Keywords: Algorithms; Longest Common Subsequence; Dynamic programming; Bit-vector algorithm

1. Introduction

Given a string x over an alphabet Σ of cardinality s , a *subsequence* of x is any string w that can be

obtained from x by deleting zero or more (not necessarily consecutive) symbols. The *Longest Common Subsequence* (LCS) problem for strings $x = x_1 \cdots x_m$ and $y = y_1 \cdots y_n$ ($n \geq m$) consists in finding a third string $w = w_1 \cdots w_p$ such that w is a subsequence of both x and y of maximum possible length. The LCS problem is related to two well-known metrics for measuring the similarity (distance) of two strings: the *Levenshtein distance* [7] and the *edit distance* [14].

The Levenshtein distance is defined as the minimum number of character insertions and/or deletions required to transform a string x into a string y . The edit distance of two strings x and y is a generalization of the Levenshtein distance where the operations allowed are symbol insertion, deletion and substitution with corresponding costs that are functions of the

* Corresponding author.

E-mail addresses: mac@univ-mlv.fr (M. Crochemore), csi@dcs.kcl.ac.uk (C.S. Iliopoulos), pinzon@dcs.kcl.ac.uk (Y.J. Pinzon), jfr@dcs.kcl.ac.uk (J.F. Reid).

URLs: <http://www-igm.univ-mlv.fr/~mac> (M. Crochemore), <http://www.dcs.kcl.ac.uk/staff/csi> (C.S. Iliopoulos), <http://www.dcs.kcl.ac.uk/pg/pinzon> (Y.J. Pinzon), <http://www.dcs.kcl.ac.uk/pg/jfr> (J.F. Reid).

¹ Partially supported by a NATO Grant and the Wellcome Trust.

² Partially supported by Royal Society, NATO and Wellcome Trust Grants.

³ Partially supported by an ORS studentship.

⁴ Supported by a Marie Curie Fellowship of the European Commission Training and Mobility of Researchers (TMR) Programme.

symbols involved. The string editing problem is that of finding the minimum cost of transforming x into y , usually called an optimal edit script. If we assign unit costs for the deletion and insertion operations and a cost of two for a substitution, then the LCS problem is directly related to the string editing problem. If d is the cost of such an optimal edit script and p is the length of an LCS then $p = 1/2(n + m - d)$.

Finding the longest common subsequence or the edit distance of two strings are problems with numerous applications ranging from text editing to molecular sequence analysis, see, e.g., [13]. A vast number of efficient algorithms have been designed over the last two decades to solve these problems. The lower bounds for the LCS problem are time $\Omega(n \log m)$ or linear time, according to whether the size of the alphabet Σ is unbounded or bounded [6]. For unbounded alphabets, any algorithm using only “equal–unequal” comparisons must take $\Omega(nm)$ in the worst case [1]. The fastest general solution for the LCS problem is the corresponding solution to the string editing problem by Masek and Paterson [9] taking $O(n^2 \log \log n / \log n)$ for unbounded alphabet size and $O(n^2 / \log n)$ for bounded alphabet size. Due to the fact that the best worst-case algorithm is still sub-quadratic in the input size, many parameterized algorithms have been designed, i.e., input- or output-sensitive algorithms.

In this paper we present a new way of computing an LCS of two strings by using bit-vector operations which is really fast in practice. The idea of using the bits of the computer word has been used extensively in the last years. In [4], Baeza-Yates and Gonnet presented an $O(nm/w)$ algorithm for the exact matching case and an $O(nm \log k/w)$ algorithm for the k -mismatches problem, where w is the number of bits in a machine word, n the length of the text and m the length of the pattern. Wu and Manber in [16] showed an $O(nkm/w)$ algorithm for the k -differences problem. Furthermore, Wright [15] presented an $O(n \log |\Sigma| m/w)$ bit-vector style algorithm where $|\Sigma|$ is the size of the alphabet for the pattern. Recently, Myers [10] developed a particularly practical method to compute the edit distance in $O(nm/w)$. Related work to the computation of the length of an LCS can be found in [2], where Allison and Dix present an $O(nm/w)$ algorithm using a bit-vector formula with five bit-wise operations (after optimization of the expression given on the paper). In this paper, we

present a similar approach to the one presented in [10] and [2] for computing the length of an LCS. The theoretical running time of the algorithm is still $O(nm/w)$ but space is $O(m/w)$ instead of $O(n/w)$ because the algorithm works in a column-wise manner instead of a row-wise one like in [2]. We also reduced the number of bit-wise operations from five to four, which is a conceptual improvement. Any efficient implementation of Allison and Dix technique benefits from our solution. Moreover, we get a simpler formula using addition instead of subtraction. The column-wise addition permits us to compute (by checking whether there is a carry at each step) the length of the LCS of the pattern p and every prefixes of the text. This can be very useful for those applications where we need to know not just the final LCS but also to keep track of the previous ones so that we can stop the computation at some point. Furthermore, adapting Myers’ algorithm for edit distance computation to solve the LCS problem only adds extra overheads.

The paper is organized as follows. In the next section we present the basic dynamic programming procedure for computing the length of an LCS, some of the properties of the resulting matrix and some definitions concerning the dominant matches. In Section 3 we describe the preprocessing and the main bit-vector algorithm for computing the length of an LCS of two strings, followed by a way to recover efficiently an actual LCS.

2. Preliminaries

Let the two input strings be $x = x_1 \cdots x_m$ and $y = y_1 \cdots y_n$ and let $L[i, j]$ denote the length of an LCS (LLCS) for the prefixes $x_1 \cdots x_i$ and $y_1 \cdots y_j$. The following simple recurrence formula by Hirschberg [5] computes $p = L[m, n]$ in $O(nm)$ time and space.

$$L[i, j] = \begin{cases} 0, & \text{if either } i = 0 \text{ or } j = 0, \\ 1 + L[i - 1, j - 1], & \text{if } x_i = y_j, \\ \max\{L[i - 1, j], L[i, j - 1]\}, & \text{if } x_i \neq y_j. \end{cases} \quad (1)$$

In fact, only linear space is needed to find the LLCS because the computation of each row/column only requires its preceding one.

		y							
		0	1	2	3	4	5	6	7
		ε	s	u	r	g	e	r	y
x	0	ε	0	0	0	0	0	0	0
	1	s	0	1	1	1	1	1	1
	2	u	0	1	2	2	2	2	2
	3	r	0	1	2	3	3	3	3
	4	v	0	1	2	3	3	3	3
	5	e	0	1	2	3	3	4	4
	6	y	0	1	2	3	3	4	5

Fig. 1. The LCS matrix L for $x = \text{"survey"}$ and $y = \text{"surgery"}$.

As an example, Fig. 1 shows the computation of the matrix L for $x = \text{"survey"}$ and $y = \text{"surgery"}$.

The LLCS p for this example is $L[6, 7] = 5$ and LCS $w = \text{"surey"}$. In this case the computed LCS is unique but that is not always the case.

Various algorithms have been designed to exploit the inherent *sparsity* of the matrix L of lengths of the LCSs of two input strings in such a way as to relate the performance of the algorithm to different parameters than the lengths of the input. Many algorithms for the LCS have used the so-called k -dominant matches. The ordered pair of positions i and j of L , denoted $[i, j]$, is a *match* iff $x_i = y_j$. If $[i, j]$ is a match, and an LCS $w_{i,j}$ of $x_1x_2 \cdots x_i$ and $y_1y_2 \cdots y_j$ has length k , then k is the *rank* of $[i, j]$. The match $[i, j]$ is k -dominant if it has rank k and for any other pair $[i', j']$ of rank k , either $i' > i$ and $j' \leq j$ or $i' \leq i$ and $j' > j$.

Computing the k -dominant matches is all that is needed to solve the LCS problem because the LCS of x and y has length p iff the maximum rank attained by a dominant match is p . We define a partial order relation on the set of matches in L by considering matches that ‘precede’ each other.

Let r be the total number of matches, and d be the total number of dominant points (of all ranks). Then $0 \leq p \leq d \leq r \leq nm$.

A match $[i, j]$ *precedes* a match $[i', j']$ if $i < i'$ and $j < j'$. Let $\mathcal{R}$ denote this partial order relation on the set of matches in L . A set of matches such that in any pair one of the matches always precedes the other in $\mathcal{R}$ constitutes a *chain* relative to the partial order relation $\mathcal{R}$. A set of matches such that in any pair neither element of the pair precedes the other in $\mathcal{R}$ constitutes an *antichain*.

Sankoff and Sellers [12] observed that the LCS problem translates to finding a longest *chain* in the poset of matches induced by $\mathcal{R}$. A decomposition of a poset into antichains partitions the poset into the minimum possible number of antichains, see, e.g., [3].

The computation of the LCS for $x = \text{"tccagatg"}$ and $y = \text{"aaagtgcactagcccg"}$ is depicted in Fig. 2. The chain from $L[1, 5]$ to $L[8, 16]$ spells out the LCS $w = \text{"tccagg"}$.

3. A simple bit-vector algorithm

Here we will make use of word-level parallelism in order to compute the matrix L more efficiently, similar to the manner used by Myers in [10] for the edit distance problem. The algorithm is based on the $O(1)$ -time computation of each column in L by using a bit-parallel formula under the assumption that $m \leq w$, where w is the number of bits in a machine word resulting in an $O(n)$ overall time complexity. In the general case, the time complexity is $O(nm/w)$ with $O(m/w)$ time per column. An interesting property of the LCS allow us to represent each column in L by using $O(1)$ -space. That is, the values in the columns (rows) of L increase by at most one. We will denote by ΔL the relative encoding of the table L , i.e., $\Delta L[i, j] = L[i, j] - L[i - 1, j] \in \{0, 1\}$ for any $(i, j) \in \{1..m\} \times \{1..n\}$. Table 1 shows the table L and the corresponding ΔL table for $x = \text{"ttgatacat"}$ and $y = \text{"gaataagacc"}$. We also define $\Delta L'$ as the negation of ΔL ($\Delta L' = \text{NOT } \Delta L$).

What $\Delta L'_j$ holds is precisely the encoding of a ‘cut’ of the k -dominant matches at position j and if

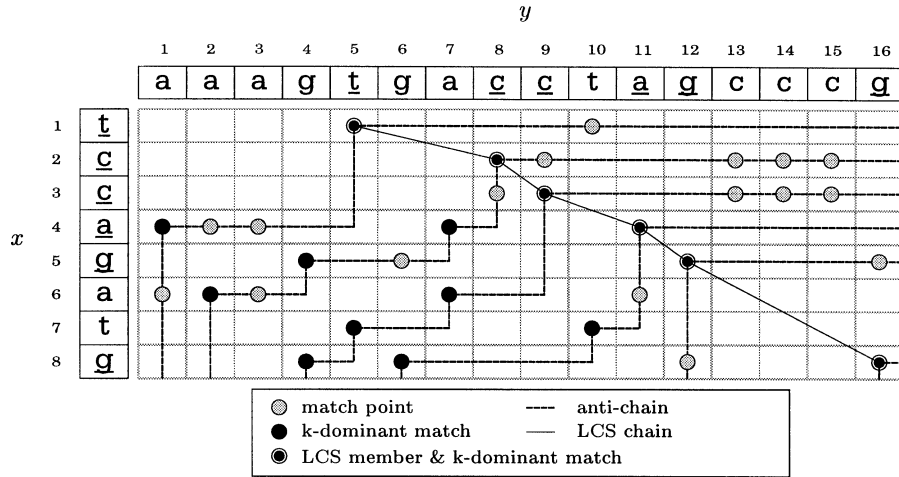
Fig. 2. LCS for $x = \text{"tccagatg"}$ and $y = \text{"aaagtacaccccg"}$.

Table 1

(a) Standard dynamic programming matrix L with k -dominant matches in bold, (b) bit-wise additive encoding ΔL of matrix L and (c) the complement of matrix ΔL ($\Delta L' = \text{NOT } \Delta L$) for $x = \text{"ttgatacat"}$ and $y = \text{"gaataagacc"}$

	0	1	2	3	4	5	6	7	8	9	10
	ϵ	g	a	a	t	a	a	g	a	c	c
0	ϵ	0	0	0	0	0	0	0	0	0	0
1	t	0	0	0	0	1	1	1	1	1	1
2	t	0	0	0	0	1	1	1	1	1	1
3	g	0	1	1	1	1	1	2	2	2	2
4	a	0	1	2	2	2	2	2	3	3	3
5	t	0	1	2	2	3	3	3	3	3	3
6	a	0	1	2	3	3	4	4	4	4	4
7	c	0	1	2	3	3	4	4	4	5	5
8	a	0	1	2	3	3	4	5	5	5	5
9	t	0	1	2	3	4	4	5	5	5	5

(a) Matrix L

	0	1	2	3	4	5	6	7	8	9	10
	ϵ	g	a	a	t	a	a	g	a	c	c
0	ϵ	0	0	0	0	0	0	0	0	0	0
1	t	0	0	0	0	1	1	1	1	1	1
2	t	0	0	0	0	0	0	0	0	0	0
3	g	0	1	1	1	0	0	0	1	1	1
4	a	0	0	1	1	1	1	0	1	1	1
5	t	0	0	0	0	1	1	1	1	0	0
6	a	0	0	0	1	0	1	1	1	1	1
7	c	0	0	0	0	0	0	0	0	1	1
8	a	0	0	0	0	0	1	1	1	0	0
9	t	0	0	0	0	1	0	0	0	0	0

(b) Matrix ΔL

	0	1	2	3	4	5	6	7	8	9	10
	ϵ	g	a	a	t	a	a	g	a	c	c
1	t	1	1	1	1	0	0	0	0	0	0
2	t	1	1	1	1	1	1	1	1	1	1
3	g	1	0	0	0	1	1	1	0	0	0
4	a	1	1	0	0	0	0	1	0	0	0
5	t	1	1	1	1	0	0	0	0	1	1
6	a	1	1	1	0	1	0	0	0	0	0
7	c	1	1	1	1	1	1	1	1	0	0
8	a	1	1	1	1	1	0	0	0	1	1
9	t	1	1	1	1	0	1	1	1	1	1

(c) Matrix $\Delta L'$

we can compute the $\Delta L'_j$ column in $O(n/w)$, then we can solve the LCS in $O(n/w)$ time. For instance, Fig. 3 illustrates how to obtain $\Delta L'_6$ by drawing a vertical line (that we called 'cut') at position 6 and we define $\Delta L'_6[i] = 0$ iff the boundary line of k -dominant matches passes at this position and 1 otherwise.

The basic steps of the algorithm are as follows:

- (1) First we compute the array M of the vectors that result for each possible text character. If both the strings x and y range over the alphabet Σ then

$M[\Sigma]$ is defined as $M[\alpha]_i = 1$ if $x_i = \alpha$ otherwise 0. This results in having a bit-vector associated to each symbol in the alphabet Σ . So for any given symbol, its vector contains 1's where a match of that symbol occurs in the string $x = x_1 \cdots x_m$. In fact from a practical point of view, we can restrict our attention to the alphabet over which the string x ranges, in case $|\Sigma| \geq m$. Here, we assume the alphabet Σ to be finite such as the ASCII machine character set. For infinite alphabets refer to [8]

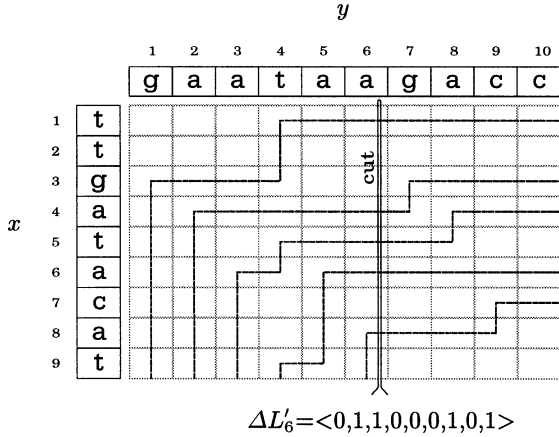


Fig. 3. The entries of the array $\Delta L'$ can be seen as ‘cuts’ across k -dominant matches boundaries in the original array L for $x = \text{“ttgatacat”}$ and $y = \text{“gaataagacc”}$.

where hashing techniques are used to overcome this problem. We also define the array M' as the negation of M . Table 2 shows the preprocessing for our example.

- (2) Let $y_j = \alpha$ be the symbol being scanned in the string y at iteration j . Let $M[\alpha]_{1:m}$ and $M'[\alpha]_{1:m}$ be the vectors resulting from the previous step on the symbol $y_j = \alpha$. Let $\Delta L'_{j-1}$ be the bit-vector column of length m of the table $\Delta L'$ computed at the previous iteration $j - 1$. We obtain the new column $\Delta L'_j$ using the following recursive formula:

$$\Delta L'_j = \begin{cases} 2^m - 1, & \text{for } j = 0, \\ (\Delta L'_{j-1} + (\Delta L'_{j-1} \text{ AND } M[y_j])) \text{ OR} \\ (\Delta L'_{j-1} \text{ AND } M'[y_j]), & \\ & \text{for } j \in \{1..n\}. \end{cases}$$

- (3) Let LLCS be the number of times that a carry took place when computing $\Delta L'_j$ in the previous step.

The algorithm in Fig. 4 computes the LLCS for two given strings x and y .

Building the array M and M' of bit-vectors can be done in $O(|\Sigma| + m)$ time and $O(|\Sigma|)$ space, assuming that $m \leq w$ or $O(|\Sigma|m/w + m)$ for the unrestricted case. The formula in the second step can be performed in $O(m/w)$ since the three bit-wise operation in the formula (AND, OR, and AND) and the addition operation can be done in $O(m/w)$ time. Therefore,

Table 2

The arrays M and M' for $x = \text{“ttgatacat”}$ and $\Sigma = \{\text{‘a’}, \text{‘c’}, \text{‘g’}, \text{‘t’}\}$

		Σ						Σ			
		a c g t						a c g t			
1	t	0	0	0	1	1	t	1	1	1	0
2	t	0	0	0	1	2	t	1	1	1	0
3	g	0	0	1	0	3	g	1	1	0	1
4	a	1	0	0	0	4	a	0	1	1	1
5	t	0	0	0	1	5	t	1	1	1	0
6	a	1	0	0	0	6	a	0	1	1	1
7	c	0	1	0	0	7	c	1	0	1	1
8	a	1	0	0	0	8	a	0	1	1	1
9	t	0	0	0	1	9	t	1	1	1	0

CIPR(x, y) $\triangleright n = |y|, m = |x|, p = 0$

```

1 begin
2    $\triangleright$  Preprocessing
3   for  $i = 1$  to  $m$  do
4      $M[\alpha]_i \leftarrow 0 \forall \alpha \in \Sigma$ 
5      $M[x_i]_i \leftarrow 1$ 
6    $M' \leftarrow \text{NOT } M$ 
7    $\triangleright$  Initialization
8    $\Delta L'_0 = 2^m - 1$ 
9    $\triangleright$  The Main Step
10  for  $j = 1$  to  $n$  do
11     $\Delta L'_j \leftarrow (\Delta L'_{j-1} + (\Delta L'_{j-1} \text{ AND } M[y_j]))$ 
12      OR  $(\Delta L'_{j-1} \text{ AND } M'[y_j])$ 
13    if  $\Delta L'_j[m + 1] = 1$  then  $p \leftarrow p + 1$ 
14       $\triangleright$  there was a carry
15  return  $p$ 
16 end
```

Fig. 4. The CIPR algorithm.

the computation of $\Delta L'_j$ for $j \in \{1..n\}$ can be done in $O(nm/w)$ time. The total overall complexity is dominated by step 2, therefore we have an $O(nm/w)$ time and $O(m/w)$ space complexity algorithm.

Our problem can also be seen as an automaton where the states are the columns of $\Delta L'$ matrix and the transition functions depends on the previous state and the preprocessing for the symbol being scanned, i.e.,

$$\Delta L'_j = f(\Delta L'_{j-1}, M[y_j]).$$

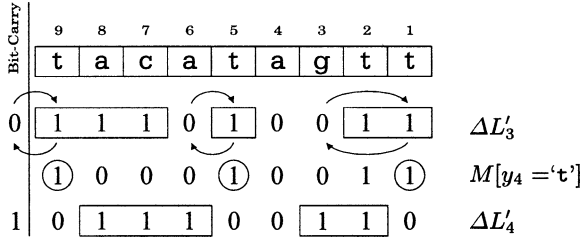


Fig. 5. Illustration of $\Delta L'_4$ for $x = \text{"ttgatacat"}$ and $y = \text{"gaataagacc"}$.

Strictly speaking

$$\Delta L'_j = f(\Delta L'_{j-1}, M[y_j], M'[y_j]).$$

Fig. 5 illustrates the computation of $\Delta L'_4$ using the previous state $\Delta L'_3$ and the preprocessing array for the symbol $y_4 = \text{'t'}$ found in $M[\text{'t'}]$. Computing the transition function f consists of two steps. In the first step we want to select the right-most bit in $M[y_j]$ for each block of 1's in $\Delta L'_{j-1}$ (see the circled numbers in Fig. 5). The second step involves swapping each of the selected bits with the closest 0 to the left in $\Delta L'_{j-1}$. In other words, the first step selects the matches that are candidates to update a current k -dominant match (0 bits in the previous state) and the second step updates them by just swapping the two. Notice that we will have a bit carry in the new state iff a new k -dominant is introduced, so if we count the number of carries (number of k -dominants introduced) we will also get the LLCS value. One could just count the number of 0's in $\Delta L'_n$. The number of bits set in a word q can be counted in $O(\log_2 q)$ (see [11]).

We now explain how the formula was obtained:

- In the first place, those matches in positions where there is a k -dominant match currently must be set off. This is obtained by *and*-ing L_{j-1} with $M[y_j]$. From our example,

$$111010011 \text{ AND } 100010011 = 100010011.$$

- Now we want to select only the right-most bit among each block of 1's and swap it with the closest 0 on its left. This is obtained by using the addition operation, which will change the left most bit for 0 and propagate this bit until a zero is found, i.e., a k -dominant match to be updated. This propagation occurs in each block simultaneously and we are ensured that the propagation is independent in each block since we have at least a zero between each

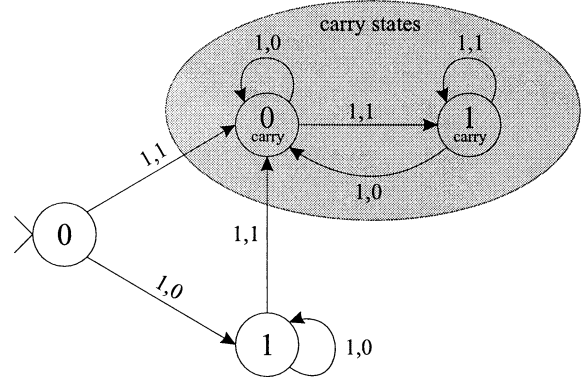


Fig. 6. Automata for the addition operation assuming that the first number is a sequence of 1's.

block. This can be easily understood by analyzing the automata for the addition operation (see Fig. 6) but in this case we will assume that the first number is a block of 1's. Then, whenever we have a bit set to 1 in the second word, (i.e., 1, 1), we will move to a state with carry (grey states). Once there it is not possible to get a state without carry. So, if there is at least a bit set to 1 in the second number then we can be sure that there will be a carry. For example,

$$111010011 + 100010011 = 011100110$$

with a bit-carry.

However, as you can see during the process there are cases where more than 1 bit is set off and we only want the right-most bit to be set off. This problem can be corrected by *or*-ing what we have so far with $(\Delta L'_{j-1} \text{ AND } M'[y_j])$. This has the effect of correcting the bits that were set off during the addition. For instance, for our example,

$$011000000 \text{ OR } 011100110 = 011100110,$$

which is exactly what we wanted to get, i.e., $\Delta L'_4$.

We implement the unbounded (any size pattern) version of CPIR algorithm in C++ and we compare its performance against Allison–Dix algorithm [2] for $t = p$ and $|\Sigma| = 26$, and the results are shown in Fig. 7. CPIR algorithm is always superior to the other for the values used in this experiment. All trials were run on a SUN Ultra Enterprise 300 MHz running Solaris Unix.

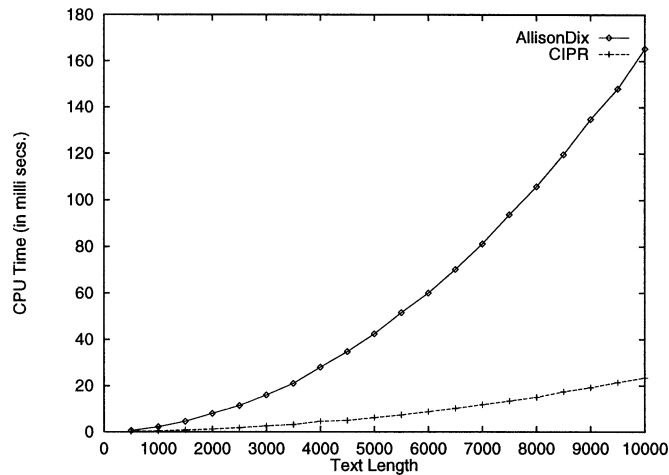


Fig. 7. Timing curves for the Allison–Dix algorithm and our CIPR algorithm.

4. Retrieving an LCS

In some applications it is required to produce an LCS instead of just its length. One option would be to use Hirschberg's paradigm [5] to recover an LCS by using divide-and-conquer together with our bit-vector algorithm for evaluating the length of an LCS. The divide-and-conquer determines the middle point of an LCS curve and then applies the procedure recursively. Each iteration uses linear space, and it can be shown that the total time used is still $O(nm/w)$.

References

- [1] A.V. Aho, D.S. Hirschberg, J.D. Ullman, Bounds on the complexity of the longest common subsequence problem, *J. ACM* 23 (1976) 1–12.
- [2] L. Allison, T.L. Dix, A bit-string longest common subsequence algorithm, *Inform. Process. Lett.* 23 (1986) 305–310.
- [3] K.P. Bogart, *Introductory Combinatorics*, Pitman, New York, 1983.
- [4] R.A. Baeza-Yates, G.H. Gonnet, A new approach to text searching, *Comm. ACM* 35 (1992) 74–82.
- [5] D.S. Hirschberg, A linear space algorithm for computing maximal common subsequences, *Comm. ACM* 18 (6) (1975) 341–343.
- [6] D.S. Hirschberg, An information theoretic lower bound for the longest common subsequence problem, *Inform. Process. Lett.* 7 (1) (1978) 40–41.
- [7] V.I. Levenshtein, Binary codes capable of correcting deletions, insertions and reversals, *Sov. Phys. Dokl.* 6 (1966) 707–710.
- [8] U. Manber, E. Myers, S. Wu, A subquadratic algorithm for approximate limited expression matching, *Algorithmica* 15 (1996) 50–67.
- [9] W.J. Masek, M.S. Paterson, A faster algorithm computing string edit distances, *J. Comput. System Sci.* 20 (1980) 18–31.
- [10] E. Myers, A fast bit-vector algorithm for approximate string matching based on dynamic programming, *J. ACM* 46 (3) (1999) 395–415.
- [11] E.M. Reingold, J. Nievergelt, N. Deo, *Combinatorial Algorithms: Theory and Practice*, Prentice-Hall, Englewood Cliffs, NJ, 1977.
- [12] D. Sankoff, P.H. Sellers, Shortcuts, diversions and maximal chains in partially ordered sets, *Discrete Math.* 4 (1973) 287–293.
- [13] D. Sankoff, J.B. Kruskal (Eds.), *Time Warps, String Edits, and Macromolecules: The Theory and Practice of Sequence Comparison*, Addison-Wesley, Reading, MA, 1983.
- [14] R.A. Wagner, M.J. Fischer, The string-to-string correction problem, *J. ACM* 21 (1) (1974) 168–173.
- [15] A.H. Wright, Approximate string matching using within-word parallelism, *Soft. Pract. Exper.* 24 (1994) 337–362.
- [16] S. Wu, U. Manber, Fast text searching allowing errors, *Comm. ACM* 35 (1992) 83–91.